

```

--
=====
=====
=====
-- Schéma Supabase
(PostgreSQL) – pmupredict
-- Ingestion multi-sources
(API tierce + scraping)
--
=====
=====
=====

-- Table des hippodromes
create table hippodromes (
    id          uuid primary
key default
gen_random_uuid(),
    code        text unique
not null,      -- ex: "M3"

```

```
(Vincennes)
    nom                text not
null,
    pays              text default
'FR',
    created_at        timestampz
default now()
);
```

```
-- Table des réunions (une
réunion = un ensemble de
courses à un hippodrome
donné)
create table reunions (
    id                uuid
primary key default
gen_random_uuid(),
    date              date not
null,
    numero            int not
null,                -- R1,
R2, ...
```

```
    hippodrome_id  uuid
references hippodromes(id),
    created_at     timestampz
default now(),
    unique (date, numero,
hippodrome_id)
);
```

```
-- Table des courses
create table courses (
    id              uuid
primary key default
gen_random_uuid(),
    reunion_id      uuid
references reunions(id),
    numero          int not
null,              -- C1,
C2, ...
    libelle         text,
    discipline      text,
-- trot, plat, obstacle
```

```
distance_m      int,  
heure_depart  
timestampz,  
etat_terrain    text,  
statut          text  
default 'a_venir',      --  
a_venir, en_cours, terminee  
created_at      timestampz  
default now(),  
unique (reunion_id,  
numero)  
);
```

```
-- Table des partants (un  
cheval dans une course  
donnée)  
create table partants (  
id              uuid  
primary key default  
gen_random_uuid(),  
course_id      uuid  
references courses(id),
```

```

    numero          int not
null,              --
numéro de corde/dossard
    cheval_nom      text not
null,
    jockey          text,
    entraineur      text,
    musique
text,
-- ex: "1a2a3a(24)0a"
    poids_kg        numeric,
    cote_matin      numeric,
    cote_actuelle   numeric,
    created_at      timestampz
default now(),
    unique (course_id, numero)
);

-- Table des arrivées /
résultats
create table arrivees (
    id              uuid

```

```

primary key default
gen_random_uuid(),
  course_id      uuid
references courses(id),
  position      int not
null,
  partant_id     uuid
references partants(id),
  ecart
text,
-- ex: "1/2 longueur"
  created_at     timestampz
default now(),
  unique (course_id,
position)
);

-- Table des prédictions IA
create table predictions (
  id              uuid
primary key default
gen_random_uuid(),

```

```

    course_id      uuid
references courses(id),
    partant_id     uuid
references partants(id),
    score          numeric not
null,
    confiance      numeric,
    modele
text,
-- ex:
"gradient_boosting_v3"
    created_at     timestampz
default now()
);

--
=====
=====
====
-- Traçabilité des sources :
la table clé pour le multi-
source

```

```

--
=====
=====
====
create table sources (
  id          uuid
primary key default
gen_random_uuid(),
  nom         text unique
not null,    -- ex:
"open_pmu_api",
"letrot_scraper"
  type       text not
null,       -- "api"
| "scraping"
  url_base   text,
  actif      boolean
default true,
  derniere_sync
timestampz,
  derniere_erreur text,
  created_at  timestampz

```

```
default now()  
);  
  
-- Trace chaque donnée :  
quelle source a alimenté  
quel enregistrement  
create table source_logs (  
    id                uuid  
primary key default  
gen_random_uuid(),  
    source_id         uuid  
references sources(id),  
    entite            text not  
null,                --  
    "course", "partant",  
    "arrivee"  
    entite_id         uuid not  
null,  
    statut            text not  
null,                --  
    "success" | "fallback" |  
    "error"
```

```
        detail          text,  
        created_at      timestampz  
default now()  
);
```

```
-- Index utiles pour les  
requêtes fréquentes  
create index  
idx_courses_reunion on  
courses(reunion_id);  
create index  
idx_partants_course on  
partants(course_id);  
create index  
idx_predictions_course on  
predictions(course_id);  
create index  
idx_source_logs_source on  
source_logs(source_id,  
created_at desc);
```

```
--
```

```
=====
=====
=====
-- RLS (Row Level Security)
-- activer et restreindre en
lecture
-- publique, écriture
réservée au service_role
(utilisé par les
-- jobs d'ingestion et l'API
backend)
--
=====
=====
=====
alter table courses enable
row level security;
alter table partants enable
row level security;
alter table arrivees enable
row level security;
alter table predictions
```

```
enable row level security;

create policy "Lecture
publique courses" on courses
    for select using (true);
create policy "Lecture
publique partants" on
partants
    for select using (true);
create policy "Ecriture
service role uniquement" on
courses
    for insert with check
(auth.role() =
'service_role');
--
=====
=====
=====
-- Registre des modèles IA
entraînés (à ajouter à
schema.sql)
```

```

--
=====
=====
=====
create table modeles (
    id                uuid
primary key default
gen_random_uuid(),
    nom                text not
null,                -- ex:
"gradient_boosting"
    version            text not
null,                -- ex:
"2026-07-26_1"
    chemin_storage     text not
null,                -- chemin
dans Supabase Storage
    metriques
jsonb,                -
- { "accuracy": 0.71,
"top5_hit_rate": 0.68, ... }
    nb_echantillons   int,

```

```

    actif          boolean
default false,    -- un
seul modèle actif à la fois
par "nom"
    created_at
timestampz default now()
);

create index
idx_modeles_actif on
modeles(nom, actif);
--
=====
=====
=====
-- Pronostics externes
(presse + communauté)
-- À ajouter à schema.sql
--
=====
=====
=====

```

```
-- Un pronostiqueur
(journaliste, site de
presse, tipster
communautaire)
create table pronostiqueurs
(
    id            uuid
primary key default
gen_random_uuid(),
    nom           text not
null,           -- ex:
"Paris-Turf", "Geny",
"tipster_jean34"
    type         text not
null,          --
"presse" | "communaute"
    site_source
text,
-- ex: "paris-turf.com"
    points       numeric
default 0,     -- score
```

```
de fiabilité cumulé (pour  
les tipsters communauté)  
  nb_pronostics  int default  
0,  
  created_at     timestampz  
default now(),  
  unique (nom, type)  
);
```

```
-- Le pronostic lui-même :  
le classement proposé par un  
pronostiqueur  
-- pour une course donnée  
create table  
pronostics_externes (  
  id                uuid  
primary key default  
gen_random_uuid(),  
  course_id         uuid  
references courses(id),  
  pronostiqueur_id  uuid  
references
```

```
pronostiqueurs(id),
  partant_id      uuid
references partants(id),
  rang_propose    int not
null,             -- 1 =
favori du pronostiqueur, 2 =
deuxième choix, etc.
  commentaire
text,             --
justification textuelle si
disponible
  source          text not
null,            -- "presse"
| "communaute" (dénormalisé
pour filtrage rapide)
  created_at
timestampz default now(),
  unique (course_id,
pronostiqueur_id,
partant_id)
);
```

```

create index
idx_pronostics_ext_course on
pronostics_externes(course_i
d);
create index
idx_pronostics_ext_pronostiq
ueur on
pronostics_externes(pronosti
queur_id);

alter table
pronostics_externes enable
row level security;
create policy "Lecture
publique pronostics
externes" on
pronostics_externes
for select using (true);
--
=====
=====
=====

```

```

-- Fusion des pronostics (IA
+ presse + communauté)
-- À ajouter à schema.sql
--
=====
=====
=====

create table
pronostics_fusion (
    id                uuid
primary key default
gen_random_uuid(),
    course_id         uuid
references courses(id),
    partant_id        uuid
references partants(id),
    score_ia          numeric
default 0,
    score_presse      numeric
default 0,
    score_communaute  numeric

```

```
default 0,  
    score_final      numeric  
not null,  
    rang_final      int not  
null,  
    poids_utilises  
jsonb,              -- ex:  
{ "ia": 0.5, "presse": 0.3,  
  "communaute": 0.2 }  
    created_at  
timestampz default now(),  
    unique (course_id,  
partant_id)  
);  
  
create index  
idx_fusion_course on  
pronostics_fusion(course_id,  
rang_final);
```